Trabajo practico 1

Introducción a los ORM

El problema del ORM

Inciso 1

¿Qué problema concreto resuelve un ORM? Describir al menos 3 (tres) inconvenientes que aparecen al intentar persistir objetos directamente con JDBC puro.

Respuesta

Un ORM es una herramienta que actúa como puente para resolver la **Diferencia de Impedancia**, permitiendo que los datos de una base de datos relacional se manipulen como objetos en el código, automatizando la traducción entre ambos mundos.

Al intentar persistir objetos mediante **JDBC puro**, aparecen los siguientes inconvenientes:

1. **Acoplamiento y falta de abstracción:** El código SQL queda embebido ("hardcodeado") dentro de la lógica de la aplicación. Esto hace que el sistema dependa de un motor de base de datos específico (dialecto), dificultando cualquier migración o cambio en la estructura de las tablas, ya que obligaría a modificar manualmente múltiples strings de consulta en todo el proyecto.
2. **Pérdida de la semántica de objetos:** En JDBC, el desarrollador se ve forzado a diseñar "modelos anémicos" o débiles. En lugar de usar asociaciones naturales (como una lista de objetos dentro de otra clase), se termina trabajando con tipos de datos primitivos o IDs numéricos para coincidir con las claves foráneas de las tablas, rompiendo los principios de encapsulamiento y polimorfismo.
3. **Gestión manual de la complejidad (Boilerplate Code):** JDBC requiere escribir una gran cantidad de código repetitivo para gestionar la apertura/cierre de conexiones, el mapeo manual de cada columna del `ResultSet` a los atributos del objeto y la conversión de tipos de datos. Esto aumenta drásticamente la probabilidad de errores en tiempo de ejecución que el compilador no puede detectar, como errores de sintaxis en los strings de SQL

Inciso 2

El modelo relacional y el modelo OO presentan tensiones conocidas como impedance mismatch. Identificar cómo se manifiesta cada una en el modelo dado

Inciso 2.1

Identidad: cómo es posible identificar un objeto Purchase en Java vs. en la base de datos?

Respuesta:

- **En Java (Objeto):** Se identifica por su **referencia de memoria** (dónde está ubicado el objeto) o por **igualdad lógica** (método `equals`). Un objeto es único porque existe en un espacio de memoria determinado.
- **En la Base de Datos (Fila):** Se identifica por su **Identidad de Datos**, específicamente a través de una **Clave Primaria (PK)**. Una compra es única porque tiene un valor (como un ID numérico) que no se repite en su tabla.
  El **ORM** resuelve esto vinculando ambos mundos: utiliza un atributo del objeto (el ID) para que la aplicación sepa qué instancia de memoria corresponde a qué fila de la tabla.

Inciso 2.2

Relaciones: cómo se navega de una Purchase a sus ItemService en Java vs. en SQL?

Respuesta:

En Java (Modelo de Objetos)

La navegación es **direccional y directa**:

- **Cómo se hace:** Se accede a través de una **referencia** o una **colección**. Si la clase `Purchase` tiene un atributo `List<ItemService> items`, simplemente haces `unaCompra.getItems()`.
- **Naturaleza:** Es una navegación "puntero a puntero". Los objetos están conectados físicamente en la memoria. Si la relación es unidireccional, solo puedes ir de la Compra al Ítem, pero no volver a menos que el objeto Ítem tenga explícitamente una referencia a la Compra.

En SQL (Modelo Relacional)
La navegación es **por valor y asociativa**:

- **Cómo se hace:** No hay un camino físico; se debe realizar una operación de **JOIN**. Para ver los ítems de una compra, debes ejecutar una consulta que compare la Clave Primaria de `Purchase` con la Clave Foránea en la tabla `Item_Service`.
- **Naturaleza:** Es bidireccional por definición. Como la relación se basa en datos (IDs), puedes consultar de Compra a Ítems o de Ítems a Compra con la misma facilidad, siempre que conozcas las claves.

Inciso 2.3

Herencia: ¿cómo podría representarse la jerarquía User/DriverUser/TourGuideUser en una tabla relacional?

Respuesta

Single Table (Una tabla por jerarquía)

Se crea una única tabla llamada `USERS` que contiene **todas** las columnas de la clase padre y de todas las clases hijas.

- **Cómo funciona:** Se agrega una columna especial llamada **Discriminador** (ej. `user_type`) para saber si esa fila es un Usuario base, un Chofer o un Guía.
- **Inconveniente:** Muchas columnas quedan en `NULL` (si una fila es un "Chofer", las columnas específicas de "Guía" estarán vacías), lo que desperdicia espacio si las hijas tienen muchos atributos distintos.

Joined (Tabla para cada clase con JOIN)

Se crea una tabla para la clase padre y una tabla para cada clase hija, conectadas por la Clave Primaria.

- **Cómo funciona:** La tabla `USERS` tiene los datos comunes (nombre, email). Las tablas `DRIVER_USERS` y `TOUR_GUIDE_USERS` solo tienen los datos específicos y su ID es, al mismo tiempo, una **Clave Foránea** que apunta a la tabla padre.
- **Inconveniente:** Para recuperar un objeto completo (ej. un Guía), la base de datos debe realizar un `JOIN`, lo que puede penalizar el rendimiento si la jerarquía es muy profunda.

Table per Class (Una tabla por cada clase concreta)

Se crean tablas independientes para cada clase que se pueda instanciar.

- **Cómo funciona:** Tendrías una tabla `DRIVER_USERS` y otra `TOUR_GUIDE_USERS`. **Cada tabla repite** las columnas de la clase padre (id, nombre, email) además de sus columnas específicas. No hay una tabla "padre" compartida en la DB.
- **Inconveniente:** Es difícil realizar consultas que involucren a toda la jerarquía (ej. "buscar todos los usuarios del sistema") porque requeriría hacer un `UNION` de todas las tablas.

Inciso 3

Describir las ventajas y desventajas concretas de usar un ORM en un proyecto como este.

Respuesta

El uso de un ORM (como Hibernate) en este proyecto resulta fundamental para gestionar la complejidad del dominio y cumplir con la arquitectura multicapa solicitada. Las principales ventajas son:

1. **Productividad y Abstracción:** Permite centrar el desarrollo en la lógica de negocio y el modelo de objetos (Rutas, Compras, Usuarios) en lugar de invertir tiempo en código JDBC repetitivo (*boilerplate*). El mapeo automático de los resultados de la base de datos a objetos Java acelera la implementación de los servicios y repositorios.

2. **Resolución de la Diferencia de Impedancia:** El ORM actúa como un puente que traduce de forma transparente las relaciones del mundo orientado a objetos (como las listas de `Stop` dentro de una `Route`) al modelo relacional, evitando tener que realizar uniones (JOINs) manuales y conversiones de tipos de datos en cada consulta.
3. **Gestión Eficiente de Relaciones (Lazy Loading):** Dado que el dominio tiene múltiples asociaciones, el ORM permite cargar colecciones solo cuando son requeridas. Esto optimiza el rendimiento y el uso de memoria, algo que en JDBC puro requeriría una lógica de carga bajo demanda muy compleja de mantener.
4. **Mantenibilidad y Portabilidad:** Al utilizar un lenguaje de consultas orientado a objetos (JPQL/HQL) y dialectos configurables, la aplicación se desacopla del motor de base de datos específico. Esto facilita futuras migraciones y asegura que los cambios en el esquema de la base de datos tengan un impacto mínimo en el código de la lógica de aplicación.
5. **Sincronización Automática de Estados:** Gracias al *Dirty Checking*, el ORM detecta automáticamente los cambios realizados en los objetos durante una transacción y genera las sentencias SQL necesarias para persistirlos al finalizar la sesión, garantizando la integridad de los datos sin intervenciones manuales constantes.

JPA y Hibernate

Inciso 4

¿Qué es JPA? ¿Qué nos permite definir? ¿Por qué se dice que es una especificación y no una implementación?

Respuesta

JPA es la **API de Persistencia de Java**, una interfaz estándar que permite a los desarrolladores gestionar datos relacionales en aplicaciones Java. Su función principal es proporcionar un marco de trabajo para el **Mapeo Objeto-Relacional (ORM)**.

JPA nos permite definir, principalmente a través de **anotaciones** (como las que estás usando en el TP), los siguientes elementos:

- **Entidades:** Clases Java que se corresponden con tablas en la base de datos (`@Entity`).
- **Identificadores:** Cuál es la clave primaria de la entidad (`@Id`) y cómo se genera (`@GeneratedValue`).
- **Relaciones:** Cómo se vinculan los objetos entre sí, definiendo la cardinalidad y la dirección (`@OneToMany`, `@ManyToOne`, `@ManyToMany`).
- **Mapeo de atributos:** Cómo se relacionan las variables de instancia con las columnas de las tablas (`@Column`).
- **Consultas:** Definir búsquedas mediante un lenguaje orientado a objetos llamado **JPQL** (Java Persistence Query Language), que opera sobre clases y no sobre tablas.

Se dice que JPA es una **especificación** porque es un "conjunto de reglas", interfaces y documentos que describen cómo debe funcionar la persistencia en Java, pero **no contiene el código que realiza el trabajo sucio** (como conectarse a la DB o ejecutar el SQL).

- **La Especificación (JPA):** Es el plano o el contrato. Define qué anotaciones usar y qué métodos deben tener los objetos (como el `EntityManager`).
- **La Implementación (Provider):** Es el motor real que ejecuta la lógica. En tu proyecto de la UNLP, **Hibernate** es la implementación (el proveedor) que "da vida" a la especificación de JPA

Inciso 5

¿Cuál es la relación entre JPA e Hibernate? ¿Puede usarse JPA sin Hibernate o Hibernate sin JPA?

Respuesta:

Relacion entre ellos:
La relación es de **Especificación e Implementación**.

- **JPA (Java Persistence API)** es el estándar o "contrato". Define las reglas, las anotaciones (como `@Entity`, `@Table`) y las interfaces (como `EntityManager`) que deben seguir las herramientas de persistencia en Java. No es un software que puedas "ejecutar" por sí solo, sino un documento que dice cómo debe funcionar el mapeo.
- **Hibernate** es el framework o "motor" que implementa ese estándar. Es el código real que traduce tus objetos a SQL y se comunica con la base de datos.

Se puede usar JPA sin hibernate, pero se necesita otro "Proveedor" (Provider). Como JPA es solo una especificación, si se desea usar Hibernate, debes elegir otra implementación que respete las mismas reglas de JPA. Ejemplos comunes son:

- EclipseLink
- OpenJPA
- TopLink

Se puede usar Hibernate sin JPA, de hecho Hibernate existía antes que JPA. Hibernate tiene su propia API nativa (la **Hibernate Native API**).

- En lugar de usar `EntityManager` (de JPA), usarías la clase `Session` de Hibernate.
- En lugar de usar `JPQL`, podrías usar `HQL` (Hibernate Query Language) puro.
- En lugar de anotaciones estándar, podrías usar archivos de configuración `.hbm.xml` o anotaciones específicas de Hibernate (org.hibernate.annotations).

Sin embargo, hoy en día no se recomienda. Lo estándar en la industria es usar Hibernate **a través de la interfaz de JPA**. Esto te da la potencia de Hibernate pero mantiene tu código

estandarizado, facilitando que otros desarrolladores lo entiendan y permitiendo cambiar de proveedor en el futuro si fuera necesario.

Inciso 6

¿Qué es la SessionFactory? ¿Qué patrón de diseño implementa? Justificar por qué se crea una sola instancia durante todo el ciclo de vida de la aplicación y no una por operación.

Respuesta

Es un objeto de Hibernate que actúa como un **depósito centralizado de configuración**. Se encarga de leer el archivo de configuración (o las anotaciones JPA), mapear las clases Java a las tablas de la base de datos y mantener el esquema de conexiones. Su función principal es crear objetos `Session`.

Implementa principalmente dos patrones:

1. **Abstract Factory (Fábrica Abstracta):** Como su nombre lo indica, es una fábrica encargada de crear instancias de `Session` de manera estandarizada.
2. **Singleton (en la práctica):** Aunque Hibernate no te obliga por código a que sea un Singleton, en todas las arquitecturas profesionales se implementa como tal para que exista una única instancia por base de datos
   Crear una `SessionFactory` es un proceso **extremadamente costoso** en términos de recursos y tiempo, por eso no se crea mas de una instancia.

Inciso 7

La SessionFactory ofrece dos formas de obtener una Session: openSession() y getCurrentSession().

Inciso 7.1

¿Cuál es la diferencia entre ambos métodos? ¿Qué ocurre con el ciclo de vida de la Session en cada caso?

Respuesta

OpenSession:

Este método siempre abre una **nueva sesión** de base de datos, independientemente de si ya existe una activa en el contexto actual.

- **Ciclo de vida:** El desarrollador es el **responsable total** del ciclo de vida. Esto significa que debes abrirla, gestionar la transacción y, lo más importante, **cerrarla explícitamente** (`session.close()`).

- **Uso común:** Se utiliza en contextos donde no hay una gestión de transacciones automática o cuando necesitas una sesión independiente para una tarea específica que no debe interferir con la principal.
- **Riesgo:** Si olvidas cerrarla, generarás una "fuga de conexiones" (*connection leak*), lo que eventualmente agotará el pool de conexiones y colapsará la aplicación.

getCurrentSession:
Este método busca una sesión que ya esté asociada al **contexto actual** (usualmente vinculado a la transacción en curso).

- **Ciclo de vida:** El ciclo de vida es **gestionado por Hibernate**. No necesitas (y de hecho no debes) cerrar la sesión manualmente. Cuando la transacción termina (ya sea con un *commit* o un *rollback*), Hibernate cierra la sesión automáticamente.
- **Requisito:** Para que funcione, hay que tener configurada la propiedad `hibernate.current_session_context_class` en el archivo de configuración (por ejemplo, con el valor `thread` o `jta`).
- **Uso común:** Es el estándar en aplicaciones modernas y frameworks como Spring ya que permite que diferentes DAOs o Repositorios compartan la misma sesión dentro de una misma unidad de trabajo

Inciso 7.2

¿Qué condición debe cumplirse para poder usar getCurrentSession()? ¿Que configuración requiere en Hibernate?

Respuesta
Para que funcione, hay que tener configurada la propiedad `hibernate.current_session_context_class` en el archivo de configuración (por ejemplo, con el valor `thread` o `jta`

Inciso 7.3

¿Quién es responsable de cerrar la Session cuando se usa getCurrentSession()? ¿Y cuando se usa openSession()?

Respuesta

getCurrentSession:
El ciclo de vida es **gestionado por Hibernate**. No necesitas (y de hecho no debes) cerrar la sesión manualmente. Cuando la transacción termina (ya sea con un *commit* o un *rollback*), Hibernate cierra la sesión automáticamente.

OpenSession:
El desarrollador es el **responsable total** del ciclo de vida. Esto significa que debes abrirla, gestionar la transacción y, lo más importante, **cerrarla explícitamente** (`session.close()`).

Inciso 7.4

¿Cuál de los dos métodos resulta más adecuado para usar en los repositorios de esta práctica y por qué?

Respuesta

El método más adecuado es `getCurrentSession()` por las siguientes razones:

- **Gestión Automática:** Al estar trabajando con **Spring**, el framework se encarga de abrir y cerrar la sesión y las transacciones de forma transparente. No necesitas cerrar la sesión manualmente, evitando fugas de conexiones.
- **Contexto Compartido:** Permite que varios repositorios participen en una misma transacción iniciada en la capa de Servicios. Todos los DAOs usarán la misma sesión para esa operación, garantizando la integridad de los datos.
- **Limpieza de Código:** Evita el código repetitivo (*boilerplate*) de apertura y cierre.

Inciso 8

Completar el siguiente cuadro

Aspecto	JPA	Hibernate
Tipo	**Especificación**	**Implementación / Framework ORM**
Define interfaces y anotaciones	**Define** el estándar (ej. `@Entity`)	**Implementa** y extiende el estándar
Proporciona implementación concreta	No (solo define las reglas)	**Sí** (es el motor que ejecuta todo)
Genera SQL a partir de consultas	Define el lenguaje **JPQL**	Traduce a **SQL** (vía HQL o JPQL)
Maneja el Persistence Context	Define el **EntityManager**	Provee la **Session**

Ciclo de vida de las entidades

Hibernate gestiona los objetos de dominio a través de un ciclo de vida bien definido. Conocerlo es
esencial para entender cuándo y cómo se persisten los cambios.

Inciso 9

Describir los cuatro estados posibles de una entidad en Hibernate (transient, managed, detached, removed) e indicar qué operación dispara cada transición entre ellos.

Respuesta

- Estado Transient (Objeto Volátil)
  - **Definición:** Son instancias que no tienen una representación en la base de datos. El libro las define como objetos que "desaparecerán cuando se cierre la aplicación". Son POJOs comunes que solo existen en la memoria RAM.
  - **Operación:** Se dispara mediante la instanciación estándar en Java: `new Clase()`.
- Estado Managed (Objeto Persistente)
  - **Definición:** Es el estado donde el objeto está vinculado al "repositorio" (base de datos) y cualquier cambio en sus atributos es monitoreado por el motor de persistencia.
  - **Operación:** * **Directo:** Invocar el método `save()` o `persist()` (en Hibernate) o `makePersistent()` (en JDO).
    - **Por Alcance (Transitivo):** Simplemente vinculando un objeto volátil a uno que ya es persistente. El libro indica que "todo objeto al cual se pueda navegar a partir de un objeto persistente, debe ser necesariamente persistente a su vez".
- Estado Detached (Objeto Desasociado)
  - **Definición:** El objeto existe en la base de datos (tiene un ID), pero la "sesión" o el "contexto" que lo controlaba se ha cerrado. Los cambios realizados en este estado no se guardan automáticamente.
  - **Operación:** Se dispara al **cerrar la sesión de trabajo** (`session.close()`) o al finalizar la transacción (`commit()`). También puede ocurrir al "limpiar" la caché del mapeador.
- Estado Removed (Objeto Eliminado)
  - **Definición:** El objeto está marcado para ser destruido en el repositorio físico.
  - **Operación:** Se dispara mediante el mensaje `delete()` o `remove()` (Hibernate) o `deletePersistent()` (JDO). También menciona que un objeto puede ser eliminado lógicamente al "desasociarlo" de sus colaboradores en un modelo puro.

Inciso 10

Describir el ciclo de vida completo de una entidad persistente, tome como ejemplo un objeto de la clase Purchase: desde que se instancia con new, pasando por su persistencia, hasta que se elimina. Indicar explícitamente en qué estado se encuentra el objeto en cada paso.

Respuesta:

Ejemplo de Ciclo de Vida: Clase `Purchase`

1. **Instanciación (`new`)**
   - **Paso:** Ejecutamos `Purchase miCompra = new Purchase();`.

- **Estado: Volátil (Transient)**.
- **Descripción según el libro:** En este momento, el objeto existe solo en la memoria RAM. Son instancias que "desaparecerán cuando se cierre la aplicación", ya que no tienen un correlato en el repositorio (base de datos).

2. **Persistencia (El paso a la base de datos)**
   - **Paso:** Vinculamos `miCompra` a un objeto que ya es persistente, por ejemplo: `miCliente.addPurchase(miCompra);`.
   - **Estado: Persistente (Managed)**.
   - **Fundamento (Pág. 73):** Aquí aplicamos la **Persistencia por Alcance**. El libro explica que no hace falta un método `save()` si el objeto es "alcanzable" desde la raíz. Al conectar la compra con el cliente, el motor de persistencia detecta el nuevo objeto y le asigna una identidad en el repositorio de forma transparente.
3. **Uso fuera de la transacción**
   - **Paso:** Se cierra la sesión o termina la unidad de trabajo.
   - **Estado: Desasociado (Detached)**.
   - **Descripción (Pág. 29):** El objeto `Purchase` sigue teniendo sus datos y su ID, pero ya no está "bajo el control" del motor. El libro menciona que en aplicaciones distribuidas, el objeto puede viajar a una interfaz gráfica, pero los cambios que se le hagan no se sincronizarán automáticamente hasta que se reabra una transacción.
4. **Eliminación**
   - **Paso:** Se quita la referencia de la compra o se solicita su borrado.
   - **Estado: Eliminado (Removed)**.
   - **Fundamento (Pág. 22):** Para eliminar un objeto en un modelo puro, este debe dejar de ser alcanzable o invocarse la operación de borrado. En este punto, el motor marca el registro para ser destruido físicamente en la base de datos durante el próximo *commit*.

Inciso 11

Investigue sobre los métodos de Session: session.save(), session.persist(), session.merge() y session.saveOrUpdate(). ¿Qué permite hacer cada uno y cuál es la diferencia entre ellos? Indicar en qué estado debe estar un objeto para usar cada uno correctamente

Respuesta

Método	Estado del Objeto	Descripción y Diferencia Clave
`save()`	**Transient**	**Propio de Hibernate.** Asigna un ID al objeto inmediatamente y lo marca para ser insertado. Este método devuelve el identificador generado .

Método	Estado del Objeto	Descripción y Diferencia Clave
`persist()`	**Transient**	**Estándar de JPA.** Similar a `save`, pero no garantiza la asignación inmediata del ID ni devuelve nada (es `void`). Está diseñado para ser usado estrictamente dentro de una transacción.
`merge()`	**Detached**	**Estándar de JPA.** Se usa para "reincorporar" cambios de un objeto que quedó fuera de la sesión. **Diferencia crucial:** No asocia el objeto original, sino que crea una copia persistente con esos datos.
`saveOrUpdate()`	**Transient** o **Detached**	**Propio de Hibernate.** Es un método "inteligente". Decide automáticamente: si el objeto es nuevo hace `INSERT`, si ya existía hace `UPDATE`.

Mapeo de Entidades

Mapeo de una entidad simple: Service

Inciso 12

12.¿Cuál es el conjunto mínimo de anotaciones que debe tener una clase para ser persistente con JPA?

Respuesta

1. `@Entity`
    1. Marca la clase Java como una entidad persistente, indicando que sus instancias tendrán una representación en la base de datos.
2. `@Id`
    1. Define cuál de los atributos de la clase será la **Clave Primaria** (Primary Key). Sin esta anotación, JPA lanzará un error al intentar arrancar la aplicación porque no puede gestionar el ciclo de vida de un objeto si no tiene una identidad única (OID).

Inciso 13

¿Qué significa que JPA use persistencia por alcance (persistence by reachability)? ¿Qué consecuencia tiene si un objeto referenciado no está todavía persistido?

Respuesta:
Significa que el motor de persistencia gestiona el ciclo de vida de los objetos basándose en su **conectividad** dentro del grafo de objetos. Un objeto se vuelve persistente de forma automática si es "alcanzable" (está referenciado) desde otro objeto que ya se encuentra en estado **Managed** (Persistente).
Si un objeto persistente (ej. `Purchase`) referencia a uno que aún es **Transient** (ej. `Product`) y

el motor no puede hacerlo persistente por alcance (ya sea por falta de configuración de cascada o por no ser un modelo de persistencia pura), se produce una **ruptura de la integridad**.
Consecuencia técnica: El motor lanzará una excepción (como `IllegalStateException` o `TransientObjectException`) al intentar realizar el *flush*, ya que no puede guardar una clave foránea hacia un registro que no existe en el repositorio físico.

Inciso 14

¿Qué diferencia hay entre las estrategias IDENTITY, SEQUENCE y TABLE para la generación de IDs? ¿Cuál tiene mejor rendimiento en inserciones masivas y por qué?

Estrategia	Funcionamiento	Dependencia
`IDENTITY`	La base de datos asigna el ID automáticamente al insertar la fila (ej. `AUTO_INCREMENT` en MySQL o `SERIAL` en PostgreSQL).	**Base de Datos**
`SEQUENCE`	Usa un objeto especial de la base de datos (secuencia) para pedir los números antes de insertar.	**Base de Datos** (Oracle, PostgreSQL)
`TABLE`	Usa una tabla separada en la BD para llevar la cuenta del último ID usado.	**Independiente** (Funciona en todas)

¿Cuál tiene mejor rendimiento en inserciones masivas?

La estrategia `SEQUENCE`.

¿Por qué?:

- **Batch Update (Inserción por lotes):** Hibernate y JPA necesitan conocer el ID del objeto **antes** de enviarlo a la base de datos para poder gestionar el grafo de objetos y la memoria caché.
- **El problema de `IDENTITY`:** Con esta estrategia, el ID solo se conoce *después* de que se ejecuta el `INSERT`. Esto obliga a Hibernate a enviar los objetos de a uno por vez, esperando que la base de datos le devuelva el ID generado. Esto **desactiva el procesamiento por lotes** (*batching*), haciendo que las inserciones masivas sean muy lentas.
- **La ventaja de `SEQUENCE`:** Permite que el motor de persistencia "pida" un rango de IDs por adelantado (por ejemplo, 50 números de una sola vez). Con los IDs ya en memoria, Hibernate puede enviar todos los `INSERT` juntos en un solo paquete (*batch*), lo que reduce drásticamente los viajes de red hacia la base de datos.

Inciso 15

Implementar el mapeo completo de la entidad Service según el diagrama. La implementación debe incluir:

1. Clave primaria con estrategia de generación automática. Elegir entre IDENTITY, SEQUENCE o TABLE y justificar la elección.
2. Atributos: name (no nulo, max. 100 caracteres), description (opcional), price (no nulo).
3. Al menos una restricción de unicidad a nivel de columna.

```
You, 11 seconds ago | 1 author (You)
@Entity
@Table(name = "services")
public class Service {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100, unique = true)
    private String name;

    @Column(nullable = false)
    private float price;

    @Column(length = 255)
    private String description;
```

Elegí la estrategia **IDENTITY** por las siguientes razones basadas en el contexto de desarrollo y la tecnología:

1. **Delegación de Responsabilidad:** Esta estrategia delega la generación del identificador directamente al motor de la base de datos (por ejemplo, usando `AUTO_INCREMENT` en MySQL o `SERIAL` en PostgreSQL).
2. **Simplicidad de Configuración:** A diferencia de `SEQUENCE` o `TABLE`, no requiere definir objetos adicionales en la base de datos (como generadores de secuencias o tablas de control de IDs). Es la opción más directa y menos propensa a errores de configuración inicial.
3. **Idoneidad para Aplicaciones Estándar:** Si bien `SEQUENCE` es superior para inserciones masivas por el soporte de *batching*, para una entidad como `Service` (donde las inserciones suelen ser puntuales y no por miles en un solo proceso), el impacto en el rendimiento es imperceptible. Por lo tanto, se prioriza la simplicidad y el soporte nativo de la mayoría de los motores relacionales.

Relaciones entre entidades

Inciso 16

Para la relación Purchase -> ItemService (composición uno-a-muchos):

1. ¿Qué anotaciones se necesitan en cada lado?
2. ¿Qué columna o tabla aparece en la base de datos para representar esta relación?
3. ¿Qué es mappedBy y en qué lado de la relación va? ¿Qué ocurre si se omite en ambos lados?
4. ¿Es esta relación bidireccional o unidireccional según el diagrama? ¿Cómo se refleja en el código Java?

Inciso 16.1

Purchase:

```
@OneToMany(mappedBy = "purchase", cascade = CascadeType.ALL, orphanRemoval =
```

ItemService:

```
@ManyToOne(optional = false)
@JoinColumn(name = "purchase_id", nullable = false)
```

Inciso 16.2

Aparece una **Columna de Clave Foránea (Foreign Key)** en la tabla `ItemService` (el lado "Muchos"). Esta columna almacena el ID de la `Purchase` a la que pertenece cada ítem.

Inciso 16.3: mappedBy

- **Qué es:** Indica que ese lado de la relación es el **inverso** (no es el dueño de la relación). Le dice a JPA: "la configuración de esta unión ya está definida en el atributo 'purchase' de la otra clase".
- **Dónde va:** Siempre en el lado **Uno** (`@OneToMany`).
- **Si se omite en ambos lados:** JPA asumirá que son dos relaciones unidireccionales independientes y creará una **tabla intermedia** (Join Table) innecesaria, lo cual es menos eficiente.

Inciso 16.4

Es **Bidireccional**. Se refleja con una colección en un lado y una referencia simple en el otro.

Inciso 17

Para las relaciones Route <-> DriverUser y Route <-> TourGuideUser (muchos-a-muchos):

1. ¿Qué anotaciones se usan?
2. ¿Qué tabla adicional genera JPA? ¿Qué columnas tiene? Definirla explícitamente usando @JoinTable.
3. ¿Pueden ambas relaciones compartir la misma tabla join? ¿Por qué?

Inciso 17.1

```
@ManyToMany
```

Inciso 17.2

JPA genera una **Tabla Intermedia (o Join Table)** para resolver la relación muchos-a-muchos, ya que el modelo relacional no permite colecciones directas. Tiene como columna la PK de Route y la PK de DriverUser, en el caso de la primer relación.

```
@ManyToMany
@JoinTable(name = "route_driver",
    joinColumns = @JoinColumn(name = "route_id"),
    inverseJoinColumns = @JoinColumn(name = "driver_user_id"))
private List<DriverUser> driverList = new ArrayList<>();      You, 19 hours ago •

@ManyToMany
@JoinTable(name = "route_tour_guide",
    joinColumns = @JoinColumn(name = "route_id"),
    inverseJoinColumns = @JoinColumn(name = "tour_guide_user_id"))
private List<TourGuideUser> tourGuideList = new ArrayList<>();
```

```
public class DriverUser extends User {
    You, yesterday • Add books, summary and fo
    @Column(nullable = false, length = 50)
    private String expedient;
    @ManyToMany(mappedBy = "driverList")
    private List<Route> routes;
```

```
You, yesterday | 1 author (You)
@Entity
public class TourGuideUser extends User {

    @Column(nullable = false, length = 100)
    private String education;

    @ManyToMany(mappedBy = "tourGuideList")
    private List<Route> routes;
```

Inciso 17.3

No, no pueden (y no deben) compartir la misma tabla.
Justificación:

1. **Semántica y Tipado:** Cada relación representa conceptos de negocio distintos. `DriverUser` y `TourGuideUser` son entidades diferentes (aunque quizás hereden de una base, tienen roles distintos). Mezclarlos en una sola tabla obligaría a tener columnas que quedarían en `null` o a usar una estructura genérica que rompe el tipado fuerte de JPA.
2. **Integridad Referencial:** Una tabla intermedia tiene claves foráneas apuntando a tablas específicas. Si intentaras usar la misma tabla, la columna `inverse_join_column` no podría apuntar simultáneamente a la tabla de Choferes y a la de Guías sin violar las restricciones de integridad de la base de datos.
3. **Mantenimiento:** Si en el futuro una ruta puede tener múltiples choferes pero solo un guía (cambia la cardinalidad), tener todo en una misma tabla haría que el refactoring sea imposible.

Inciso 18

La relación Purchase -> Review es opcional (0..1). Implementar el mapeo de ambos lados. ¿Cómo se representa la opcionalidad en JPA?

Respuesta:
En JPA, la opcionalidad se representa en el lado dueño de la relación (el que tiene la FK) con:

- optional en la anotación de relación.
- nullable en JoinColumn.
  Para Purchase 0..1 Review, el mapeo bidireccional típico es:

En Purchase (lado inverso):

```
@OneToOne(mappedBy = "purchase")
```

```
private Review review;
```

En Review (lado dueño, con FK):

```
@OneToOne(optional = false)
@JoinColumn(name = "purchase_id", nullable = false, unique = true)
```

Inciso 19

ItemService referencia a Service (muchos-a-uno). Analizar el diagrama: ¿es navegable esta relación desde Service hacia ItemService? Justificar si conviene hacerla bidireccional o no.

Respuesta

Sí, según el diagrama se puede interpretar como navegable desde Service hacia ItemService, porque aparece la colección de ítems asociada al servicio (cardinalidad 1 a *).

Si conviene hacerla bidireccional o no:

- Conviene bidireccional si en tus casos de uso necesitás consultar desde un servicio sus ítems (por ejemplo, métricas, reportes, auditoría de uso).
- Conviene unidireccional (solo ItemService -> Service) si casi nunca navegás desde Service, para simplificar modelo, evitar sincronizar dos lados y reducir riesgos de ciclos al serializar.

Inciso 20

Implementar el mapeo completo de las siguientes entidades con todas sus relaciones, siguiendo las anotaciones y decisiones discutidas: Supplier, Purchase, ItemService, Route, Stop y Review. Para cada relación bidireccional, incluir las anotaciones en ambos lados.

Respuesta

```
You, yesterday | 1 author (You)
@Entity
@Table(name = "suppliers")
public class Supplier {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String businessName;

    @Column(nullable = false, length = 100)
    private String authorizationNumber;

    @OneToMany(mappedBy = "supplier")
    private List<Service> services;        You, yesterday • Add books, summary and fol

    public Supplier() {
    }

    public Supplier(String businessName, String authorizationNumber) {
        this.businessName = businessName;
        this.authorizationNumber = authorizationNumber;
    }
```

```
You, 9 minutes ago | 1 author (You)
@Entity
@Table(name = "purchases")
public class Purchase {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String code;

    @Column(nullable = false)
    private float totalPrice;

    @Column(nullable = false)
    @Temporal(TemporalType.DATE)
    private Date date;

    @ManyToOne(optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @ManyToOne(optional = false)
    @JoinColumn(name = "route_id", nullable = false)
    private Route route;

    @OneToOne(mappedBy = "purchase", cascade = CascadeType.ALL, orphanRemoval = true)
    private Review review;

    @OneToMany(mappedBy = "purchase", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)
    private List<ItemService> itemServiceList = new ArrayList<>();

    public Purchase() {
    }
      You, yesterday • Add books, summary and folders to pers to pract…
```

```
You, now | 1 author (You)
@Entity
@Table(name = "item_services")
public class ItemService {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private int quantity;

    @ManyToOne(optional = false)
    @JoinColumn(name = "purchase_id", nullable = false)
    private Purchase purchase;

    @ManyToOne(optional = false)
    @JoinColumn(name = "service_id", nullable = false)
    private Service service;

    public ItemService() {
    }

```

You, 20 hours ago | 1 author (You)

```java
@Entity
@Table(name = "routes")
public class Route {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false)

    private float price;
    @Column(nullable = false)
    private float totalKm;

    @Column(nullable = false)      You, yesterday • Add annotations to classes and gitignore for ja…
    private int maxNumberUsers;

    @ManyToMany
    @JoinTable(name = "route_stop",
        joinColumns = @JoinColumn(name = "route_id"),
        inverseJoinColumns = @JoinColumn(name = "stop_id"))
    private List<Stop> stops = new ArrayList<>();

    @ManyToMany
    @JoinTable(name = "route_driver",
        joinColumns = @JoinColumn(name = "route_id"),
        inverseJoinColumns = @JoinColumn(name = "driver_user_id"))
    private List<DriverUser> driverList = new ArrayList<>();

    @ManyToMany
    @JoinTable(name = "route_tour_guide",
        joinColumns = @JoinColumn(name = "route_id"),
        inverseJoinColumns = @JoinColumn(name = "tour_guide_user_id"))
    private List<TourGuideUser> tourGuideList = new ArrayList<>();

    public Route() {
    }
```

You, yesterday | 1 author (You)

```java
@Entity
@Table(name = "stops")
public class Stop {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, length = 255)
    private String description;

    @ManyToMany(mappedBy = "stops")
    private List<Route> routeList;

    public Stop() {
    }

```

```java
You, yesterday | 1 author (You)
@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private int rating;

    @Column(length = 255)
    private String comment;

    @OneToOne(optional = false)      You, yesterday • Add annotations to classes and gitignore for ja…
    @JoinColumn(name = "purchase_id", nullable = false, unique = true)
    private Purchase purchase;

    public Review() {
    }
    public Review(int rating, String comment, Purchase purchase) {
        this.rating = rating;
        this.comment = comment;
        this.purchase = purchase;
    }
```

Fetch Type: LAZY vs. EAGER

Inciso 21

¿Para qué sirve la propiedad fetch en las anotaciones de relación? ¿Cuáles son los valores posibles? ¿Cuál es el valor por defecto para @OneToMany, para @ManyToOne y para @ManyToMany?

Respuesta:
Sirve para definir la **estrategia de carga** de los objetos relacionados. Determina si los datos de las asociaciones se deben traer de la base de datos al mismo tiempo que el objeto principal o si deben cargarse únicamente cuando se acceda a ellos por primera vez.
Valores posibles

- `FetchType.LAZY` **(Carga diferida):** No recupera los objetos relacionados de inmediato. Crea un "proxy" y solo realiza la consulta a la base de datos cuando se invoca el método *getter* de la relación.
- `FetchType.EAGER` **(Carga temprana):** Recupera el objeto principal y todos sus relacionados en la misma consulta (usualmente mediante un `JOIN`), cargando todo en memoria RAM desde el inicio.

Anotación	Valor por defecto	Explicación
@OneToMany	LAZY	Como "muchos" objetos pueden ser miles, JPA no los trae por defecto para no saturar la memoria.
@ManyToMany	LAZY	Al igual que el anterior, al ser una colección, se asume carga diferida

Anotación	Valor por defecto	Explicación
@ManyToOne	EAGER	Como la relación apunta a **un solo** objeto padre, JPA lo trae por defecto porque el costo de rendimiento es bajo.

Inciso 22

Describir ventajas y desventajas concretas de EAGER y LAZY, en términos de performance de acceso como de espacio en memoria. ¿Por qué configurar EAGER en todas las relaciones suele ser una mala idea en aplicaciones reales?

Respuesta:

- **. EAGER (Carga Temprana)**
  - **Ventajas:**
    - **Disponibilidad Inmediata:** Los datos están listos en memoria. No hay riesgo de `LazyInitializationException` una vez cerrada la sesión.
    - **Menos viajes a la BD (a veces):** Si realmente necesitás los datos, JPA suele usar un `LEFT JOIN` para traer todo en una sola consulta SQL, ahorrando latencia de red.
  - **Desventajas:**
    - **Consumo de Memoria:** Se instancian objetos en la RAM que quizás nunca se usen.
    - **Performance Inicial:** La consulta inicial es más pesada y lenta debido a los múltiples `JOINs` que debe procesar el motor de base de datos.

- LAZY (Carga Diferida)
  - **Ventajas:**
    - **Eficiencia de Memoria:** Solo se ocupa espacio para los objetos estrictamente necesarios.
    - **Arranque Rápido:** La consulta inicial es liviana y veloz (trae solo los datos de la tabla principal).
  - **Desventajas:**
    - **Problema de las "N+1 Consultas":** Si listás 100 compras y luego querés ver sus ítems, JPA hará 1 consulta para las compras y **100 consultas adicionales** para los ítems (una por cada compra), lo que liquida la performance de red.
    - **Dependencia de la Sesión:** Si intentás acceder a los datos fuera del contexto transaccional (ej. en la capa de vista), la aplicación fallará.

Configurar EAGER en todas las relaciones genera un **acoplamiento excesivo** en la recuperación de datos. Provoca un uso ineficiente de la memoria RAM al cargar grafos de objetos innecesarios y degrada la performance de la base de datos al ejecutar consultas con

excesivos `JOINs`. En aplicaciones reales, se prefiere LAZY por defecto para mantener la aplicación escalable y liviana.

Inciso 23

Para cada relación del modelo, elegir el FetchType más adecuado y justificar. Luego implemente su decisión en el proyecto tours.

Relación	Anotación	Fetch	Justificación
Purchase -> User	`@ManyToOne`	**Eager**	Al ser una referencia a un único objeto, el costo de carga es mínimo y permite tener los datos del comprador siempre disponibles.
Purchase -> Route	`@ManyToOne`	**Eager**	La ruta es un dato esencial de la compra. Al ser una relación simple (1-1 en la instancia), se recupera eficientemente con un `JOIN`.
Purchase -> items	`@OneToMany`	**Lazy**	Una compra puede tener muchos ítems. Se usa Lazy para evitar cargar grandes colecciones en memoria si solo se necesita consultar datos básicos de la compra.
Purchase -> Review	`@OneToOne`	**Lazy**	**Diferencia clave:** Aunque es uno a uno, la reseña es opcional y no siempre se consulta junto con la compra. Usar Lazy evita cargar texto pesado innecesariamente.
ItemService -> Service	`@ManyToOne`	**Eager**	Para procesar un ítem de servicio, es indispensable conocer qué servicio es (nombre, precio). La carga temprana facilita el acceso directo a estos datos.
Route -> stops	`@OneToMany`	**Lazy**	Las paradas de una ruta pueden ser numerosas. Se prefiere carga diferida para no penalizar la performance al listar rutas.
Route -> drivers	`@ManyToMany`	**Lazy**	Las relaciones muchos-a-muchos implican tablas intermedias. El costo de procesamiento es alto, por lo que solo se cargan si se requiere administrar el personal.
Route -> guides	`@ManyToMany`	**Lazy**	Ídem anterior. Evita traer todo el grafo de usuarios (Guías) cada vez que se consulta una disponibilidad de ruta.
User -> purchases	`@OneToMany`	**Lazy**	Un usuario puede tener un historial de compras muy extenso. Cargar todas las compras al loguear al usuario colapsaría la memoria RAM.
Service -> supplier	`@ManyToOne`	**Eager**	El proveedor es un dato atómico del servicio. Al ser una relación `@ManyToOne` hacia una sola entidad, el impacto en performance es despreciable.

Inciso 24

¿Cómo podría producirse una LazyInitializationException en el modelo? Investigue de qué representa esta excepción y escribir un escenario concreto explicando al menos formas de resolverlo sin cambiar el FetchType a EAGER.

Respuesta:
Representa un intento de acceder a una asociación o colección marcada como **LAZY** cuando la **Sesión de Hibernate (el EntityManager)** ya ha sido cerrada.

En JPA, los objetos "Lazy" no son los datos reales, sino **Proxies** (objetos huecos). Para llenarse con datos reales, necesitan pedirle permiso a la base de datos a través de una conexión activa. Si la conexión (sesión) se cerró, el Proxy no tiene a quién pedirle los datos y lanza la excepción.

Escenario concreto:
Imaginá que estás en tu proyecto de viajes y tenés la relación `Purchase -> itemServiceList` como **LAZY**.

1. **Capa de Servicio:** Llamás a `purchaseDAO.getById(1)`. Hibernate abre la sesión, trae los datos básicos de la `Purchase` y **cierra la sesión** al terminar el método del servicio.
2. **Capa de Controlador/Presentación:** Querés mostrar el detalle en pantalla y hacés:

```
System.out.println(unaPurchase.getItemServiceList().size());
```

1. **Resultado:** Como la sesión se cerró en el paso 1, al intentar "navegar" hacia la lista de ítems en el paso 2, se produce la `LazyInitializationException`.

Cómo resolverlo (sin usar EAGER):

Uso de `JOIN FETCH` (La solución más profesional)

En lugar de un `findAll()` genérico, creás una consulta personalizada en tu DAO usando JPQL que fuerce la carga de esa relación específica solo para ese caso.

- **Consulta:** `SELECT p FROM Purchase p JOIN FETCH p.itemServiceList WHERE p.id = :id`
- **Por qué funciona:** El `FETCH` le dice a Hibernate: "Traé la compra y, *en esta consulta puntual*, traé también los ítems aunque sean Lazy". Así, cuando la sesión se cierra, la lista ya está llena.

B. Inicialización forzada (Hibernate.initialize)

Dentro del método del Servicio (mientras la sesión sigue abierta), podés "despertar" al Proxy manualmente.

- **Código:** `Hibernate.initialize(unaPurchase.getItemServiceList());`
- **Por qué funciona:** Obliga a Hibernate a realizar la consulta select de los ítems antes de que el método termine y se cierre la conexión.

C. Patrón "Open Session in View" (OSIV)

Es una configuración del framework (como Spring Boot) que mantiene la sesión de base de datos abierta hasta que la respuesta llega al navegador (atraviesa toda la capa de vista).

- **Pros:** Te olvidás del error para siempre.
- **Contras:** Es polémico porque mantiene conexiones a la base de datos abiertas más tiempo del necesario, lo que puede afectar la escalabilidad en aplicaciones muy grandes.

Operaciones en cascada

Inciso 25

Enumerar todos los valores de CascadeType y explicar qué operación propaga cada uno sobre las entidades relacionadas.

Respuesta:

1. `PERSIST`
   - **Operación:** `EntityManager.persist()` o `session.persist()`.
   - **Explicación:** Si creás una entidad nueva y la guardás, todas las entidades relacionadas que estén en la colección o referencia también se guardarán automáticamente en la base de datos. Evita tener que hacer `save` uno por uno.
2. `REMOVE`
   - **Operación:** `EntityManager.remove()` o `session.delete()`.
   - **Explicación:** Si eliminás la entidad padre, JPA eliminará automáticamente todos los registros relacionados. Es fundamental para mantener la integridad referencial y no dejar "datos huérfanos" en las tablas.
3. `MERGE`
   - **Operación:** `EntityManager.merge()` o `session.update()`.
   - **Explicación:** Si recuperás una entidad desasociada (**Detached**), le modificás datos (a ella o a sus hijos) y llamás a `merge`, JPA actualizará el estado de todo el grafo de objetos en la base de datos.
4. `REFRESH`
   - **Operación:** `EntityManager.refresh()`.

- **Explicación:** Sincroniza el estado del objeto en memoria con los datos actuales de la base de datos. Si el padre se refresca, todos sus hijos también volverán a leer sus valores desde las tablas, descartando cambios no persistidos en memoria.

5. `DETACH`
   - **Operación:** `EntityManager.detach()`.
   - **Explicación:** Quita las entidades del contexto de persistencia (pasan a estado **Detached**). Al aplicarlo en cascada, tanto el padre como sus hijos dejan de estar "trackeados" por Hibernate, lo que libera memoria pero impide que los cambios se guarden automáticamente hasta un próximo `merge`.
6. `ALL`
   - **Operación:** Todas las anteriores.
   - **Explicación:** Es un atajo que engloba a todos los tipos de cascada. Es el más común en relaciones de **composición fuerte**, donde el ciclo de vida del hijo está totalmente atado al del padre.

Conclusión:
La cascada permite automatizar la gestión del ciclo de vida del grafo de objetos. El valor más seguro para composiciones es **ALL**, mientras que para asociaciones simples se recomienda ser específico (usar solo **PERSIST** o **MERGE**) para evitar borrados accidentales de entidades que podrían ser compartidas por otros objetos

Inciso 26

¿Cuál es el comportamiento por defecto cuando no se define cascade? ¿Cuál es la finalidad general de CASCADE? Proponga un caso del modelo donde definir un CASCADE inadecuado podría traer problemas a la consistencia de la base de datos.

Respuesta

Comportamiento por defecto:

- Cuando no se define la propiedad `cascade`, el comportamiento es **nulo o restrictivo**.
  - Si intentás guardar un objeto que referencia a otro objeto nuevo (Transient), JPA lanzará una excepción (`IllegalStateException` o `TransientObjectException`) porque no tiene permiso para persistir al objeto relacionado automáticamente.
  - Si borrás el objeto padre, la relación en la base de datos simplemente se romperá (si la FK permite nulos) o lanzará un error de restricción de integridad, pero **no borrará** al objeto hijo.

La finalidad es la **automatización del ciclo de vida del grafo de objetos**. Permite que las operaciones de persistencia (guardar, actualizar, borrar) se propaguen de forma transitiva a través de las asociaciones. Esto garantiza la **integridad referencial** sin que el programador tenga que realizar múltiples llamadas manuales

Inciso 27

¿Cuál es la diferencia entre cascade = REMOVE y orphanRemoval = true? ¿Pueden usarse conjuntamente? Ejemplificar con el par Purchase -> ItemService.

Diferencia principal

Característica	cascade = REMOVE	orphanRemoval = true
Gatillo (Trigger)	Se activa al borrar el **padre**.	Se activa al sacar al **hijo** de la colección del padre.
Acción	Si hacés `session.remove(purchase)`, se borran sus ítems.	Si hacés `purchase.getItems().remove(item)`, se borra ese ítem.
Concepto	Propagación de una operación de borrado explícita.	Limpieza automática de "huérfanos" (objetos sin referencia).

Uso en conjunto:
Sí, y en una composición es lo ideal. De hecho, en el par Purchase -> ItemService** se recomienda usar ambos (o `CascadeType.ALL` que incluye REMOVE + `orphanRemoval = true`).

Al combinarlos, garantizás que la base de datos esté siempre limpia:

- Si borrás la factura, mueren todos los ítems (**Cascade**).
- Si editás la factura y sacás un renglón de la lista porque el cliente se arrepintió, ese renglón se borra de la tabla automáticamente (**Orphan Removal**).

Conclusión:
`cascade = REMOVE` se encarga del borrado "en bloque" (padre e hijos), mientras que `orphanRemoval` se encarga del borrado por "desvinculación" (sacar un hijo de la lista). En una **composición**, usarlos juntos es la única forma de asegurar que un `ItemService` nunca exista sin su `Purchase`

Inciso 28

Para la relación Purchase -> ItemService (composición):

1. ¿Qué tipos de cascade configurarías? Justificar cada uno.
2. ¿Usarias orphanRemoval? ¿Por qué?
3. Describir qué ocurre a nivel de base de datos cuando se elimina un ItemService de la lista de una Purchase y Hibernate realiza el flush.

Inciso 28.1

Lo ideal es configurar `PERSIST`, `MERGE` y `REMOVE` (o directamente `ALL`).

- **Justificación de `PERSIST`**: Permite que al guardar una nueva `Purchase`, todos sus `ItemService` se guarden automáticamente. Evita tener que persistir cada renglón de la factura manualmente.
- **Justificación de `MERGE`**: Esencial para actualizaciones. Si editás una compra (por ejemplo, cambiás una cantidad en un ítem) y llamás a `update`, los cambios se propagan a los ítems. Sin esto, Hibernate ignoraría las modificaciones en los hijos.
- **Justificación de `REMOVE`**: Al ser una composición, el `ItemService` no tiene razón de existir sin su `Purchase`. Si se elimina la compra, los ítems deben borrarse para no dejar registros "huérfanos" que ensucien la base de datos.

Inciso 28.2

Sí, usaría `orphanRemoval = true`.

- **Justificación**: En una factura, si decidís quitar un renglón de la lista (por ejemplo, `compra.getItems().remove(unItem)`), ese objeto deja de tener sentido en el dominio del negocio. Activar esta propiedad le indica a Hibernate que, al romperse el vínculo con el padre, debe ejecutar un **borrado físico** del registro en la tabla `item_service`.

Inciso 28.3

Cuando se elimina el objeto de la lista en Java y Hibernate sincroniza los cambios con la base de datos (**flush**):

1. **Detección de desvinculación**: Hibernate nota que un `ItemService` que antes estaba asociado a la `Purchase` ya no figura en la colección.
2. **Ejecución de SQL**: Gracias a `orphanRemoval`, se dispara automáticamente una sentencia `DELETE FROM item_service WHERE id = ?`.
3. **Integridad del Catálogo**: El registro en la tabla `service` (el servicio base como "City Tour") **no se ve afectado**. Solo se elimina el registro de la venta específica (el renglón), pero el producto del catálogo permanece intacto porque no existe una relación de cascada hacia él.

Inciso 29

Para la relación Purchase -> Review:

1. ¿Qué cascades configurarías?
2. Si se elimina una Purchase, ¿debería eliminarse también su Review? Justificar desde el modelo de negocio.

Inciso 29.1

Configuraría `CascadeType.MERGE` y `CascadeType.REMOVE`.

- `MERGE`: Si el usuario edita algún dato de la compra (como el estado) y al mismo tiempo la reseña ya existe, queremos que cualquier cambio en el grafo de objetos se sincronice. Aunque en un `@OneToOne` los cambios suelen ser independientes, el `MERGE` asegura que no perdamos la vinculación si el objeto llega desasociado del frontend.
- `REMOVE`**: Es vital para mantener la** integridad referencial**. Si eliminamos el registro de la compra, la reseña quedaría apuntando a un ID inexistente o quedaría como un dato huérfano sin contexto. Propagar el borrado asegura que la base de datos se mantenga limpia.
- **¿Por qué NO `PERSIST`?**: (Opcional) A diferencia de los ítems, una reseña suele crearse **mucho después** de la compra. Si ponés `PERSIST`, podrías crear accidentalmente reseñas vacías al guardar la compra por primera vez. Es mejor manejar la creación de la `Review` como un evento separado.

Inciso 29.2

Sí, definitivamente.

1. **Dependencia Existencial**: Una reseña es un comentario subjetivo sobre una experiencia de consumo específica. Si la transacción (la compra) se anula o se elimina del sistema por motivos de privacidad o error, el comentario pierde su **referente real**. No se puede reseñar "la nada".
2. **Consistencia de Datos**: Mantener reseñas de compras inexistentes distorsionaría las métricas de satisfacción del sistema (por ejemplo, el puntaje promedio de un `Service` o `Route`), ya que estaríamos contando votos de operaciones que legal o técnicamente ya no figuran en los registros.
3. **Privacidad (RGPD/Derecho al olvido)**: Si un usuario pide borrar su historial de compras, por extensión se entiende que desea eliminar su actividad vinculada a esas compras, incluyendo las opiniones vertidas.

Inciso 30

Para la relación Supplier -> Service:

1. ¿Qué cascades tienen sentido?
2. Si se elimina un Supplier, ¿qué debería ocurrir con sus Service? ¿Y con las Purchase que los contienen a través de ItemService?

Inciso 30.1

Para la relación `Supplier -> Service` (`OneToMany`), solo tienen sentido `PERSIST` y `MERGE`.

- **Justificación de `PERSIST`**: Permite que, al dar de alta a un nuevo Proveedor, puedas cargar de un solo tirón su catálogo de servicios inicial.

- **Justificación de `MERGE`**: Si el Proveedor actualiza las condiciones de sus servicios (precios, descripciones), el cambio se propaga correctamente desde el objeto raíz `Supplier`.
- **¿Por qué NO `REMOVE`?**: (Crucial) Un `Supplier` es una entidad independiente. Borrar un proveedor no debería significar borrar sus servicios de la faz de la tierra si esos servicios ya fueron vendidos a clientes.

Inciso 30.2

Para la relación **Supplier -> Service**, **NO se debe configurar `REMOVE`**. Si se elimina un `Supplier`, sus `Service` deberían permanecer en la base de datos (quizás asociados a un 'Proveedor Genérico' o marcados como inactivos) para no romper la integridad de las `Purchase` ya realizadas. Las compras son documentos históricos que no deben alterarse ni eliminarse por cambios en el catálogo de proveedores

Inciso 31

¿Por que CascadeType.REMOVE en una relación @ManyToMany (por ejemplo Route <-> DriverUser) suele ser peligroso? Describir un escenario donde su uso cause pérdida no deseada de datos.

Respuesta:

En `@ManyToMany`, `CascadeType.REMOVE` es extremadamente peligroso porque **asume que el objeto hijo es propiedad exclusiva del padre**, lo cual es falso en una relación muchos-a-muchos. Su uso provoca la eliminación en cascada de entidades compartidas, rompiendo la integridad de otras relaciones en el sistema. En estas asociaciones, siempre se debe preferir el manejo manual del vínculo (borrar solo la fila en la tabla intermedia) en lugar de permitir que la cascada borre la entidad del otro lado de la relación

Escenario concreto:

Imaginemos el siguiente escenario

- **Estado:** Tenemos al `DriverUser` llamado "Juan Pérez".
- **Relación:** "Juan Pérez" está asignado a:
  - `Ruta A` (La Plata - Buenos Aires)
  - `Ruta B` (La Plata - Córdoba)
- **Acción:** Un administrativo decide eliminar la `Ruta A` porque ya no se opera.
- **El problema:** Debido al `CascadeType.REMOVE` configurado:
  1. Hibernate detecta que "Juan Pérez" estaba en la `Ruta A`.
  2. Procede a ejecutar `DELETE FROM driver_user WHERE id = 'JuanPerezID'`.
  3. **Pérdida:** "Juan Pérez" desaparece de la base de datos.

4. **Resultado no deseado:** Cuando los encargados de la `Ruta B` intentan consultar quién es el chofer de esa ruta, el sistema falla o muestra que el chofer ya no existe. Borraste a una persona de toda la empresa solo porque cancelaste uno de sus muchos trayectos.

Inciso 32

Implemente las operaciones en cascada adecuada para todas las relaciones del modelo tours.

Anotaciones como justificación:

Relación User -> Purchase (@OneToMany)
Decisión: cascade = {PERSIST, MERGE, REFRESH, DETACH} (sin REMOVE, sin ALL).

CascadeType	Operación	Justificación Técnica
PERSIST	`persist(user)`	Permite persistir un usuario nuevo junto con sus compras iniciales en una única operación transaccional, sin necesidad de llamadas manuales al DAO por cada compra.
MERGE	`merge(user)`	Si el objeto llega en estado *detached* (ej. desde el frontend), sincroniza los cambios tanto en los datos del usuario como en su lista de compras de forma consistente.
REFRESH	`refresh(user)`	Garantiza que, al refrescar el estado del usuario desde la base de datos, sus compras también se actualicen, evitando inconsistencias si hubo cambios concurrentes.
DETACH	`detach(user)`	Al quitar al usuario del contexto de persistencia, se desacoplan también sus compras, reduciendo el tracking en memoria y evitando actualizaciones accidentales.

Relación Supplier -> Service (@OneToMany)
Decisión: cascade = {PERSIST, MERGE, REFRESH, DETACH} (sin REMOVE, sin ALL).

CascadeType	Operación	Justificación Técnica
PERSIST	`persist(supplier)`	Facilita la creación de un proveedor y la carga de su catálogo de servicios inicial en un solo paso, manteniendo la atomicidad de la operación.
MERGE	`merge(supplier)`	Permite actualizar la información del proveedor y las condiciones de sus servicios asociados (precios, descripciones) cuando el objeto se encuentra fuera de la sesión.

Relaciones Route -> Stop, Route -> DriverUser, Route -> TourGuideUser (@ManyToMany)
Decisión: cascade = {PERSIST, MERGE} (sin REMOVE, sin REFRESH, sin DETACH, sin ALL).

CascadeType	Operación	Justificación Técnica
PERSIST	`persist(route)`	Si se define una nueva ruta con paradas o personal (Choferes/Guías) también nuevos, se persisten todas las entidades vinculadas sin requerir persistencias manuales repetitivas.
MERGE	`merge(route)`	Al actualizar una ruta *detached* (ej. reasignación de un chofer o cambio de una parada), el merge sincroniza tanto la relación en la tabla intermedia como los cambios en los objetos vinculados.

- **Sin REMOVE en `@ManyToMany`:** No se aplica para evitar la pérdida de "entidades compartidas". Borrar una ruta no debe eliminar al chofer de la base de datos, ya que este puede estar asignado a otras rutas.
- **Sin REMOVE en Supplier:** Se omite para proteger la **integridad histórica**. Los servicios de un proveedor no pueden borrarse si existen compras (`ItemService`) que los referencian.
- **Uso de REFRESH/DETACH:** Se mantienen en las relaciones `@OneToMany` de usuarios y proveedores porque ayudan a gestionar grafos de objetos grandes en memoria, pero se omiten en las `@ManyToMany` de rutas para mantener la operación lo más liviana y específica posible.